```
Scenario: Logging on with an expired account
  Given the account has expired
  When Martin logs on with password 'secret'
  Then he should be informed that his account has expired
  And he should be invited to renew his account
```

The `Background` keyword lets you specify steps that will be run before each scenario in the feature. You can use this to avoid duplicating steps in each scenario, which also helps focus attention on the important bits of each scenario.

In JBehave, you can do something similar with the `GivenStories` keyword, though it's a bit more technical. This keyword lets you run the scenarios in arbitrary `.story` files either before each scenario or once before any of the scenarios are executed. For example, you could place the background steps in a file called `Martin_has_registered.story`:

```
Scenario: Registering for the site
Given Martin is a Frequent Flyer member
And Martin has registered online with a password of 'secret'
```

This goes in a separate .story file.

Then you would modify your main `.story` file to refer to the preceding file using the `GivenStories` keyword, as follows.

Listing 5.2 A JBehave story using the `GivenStories` keyword

```
Logging on to the 'My Flying High' website

Scenario: Logging on successfully
GivenStories: Martin_has_registered.story
When Martin logs on with password 'secret'
Then he should be given access to the site

Scenario: Logging on with an incorrect password
GivenStories: Martin_has_registered.story
When Martin logs on with password 'wrong'
Then he should be informed that his password was incorrect

Scenario: Logging on with an expired account
GivenStories: Martin_has_registered.story
Given the account has expired
When Martin logs on with password 'secret'
Then he should be informed that his account has expired
And he should be invited to renew his account
```

5.4.5 Avoid dependencies between scenarios

It's important to remember that even if the scenarios live together in the same feature file, each scenario should be able to work in isolation. A scenario should not depend on a previous one running to set up data or to put the system into a particular state.

For example, suppose you're writing scenarios for an e-commerce site. You might write something like this:

```
Scenario: Adding an item to the cart
    Given I select 'BDD in Action'
    When I add it to the cart
    Then my cart should contain one copy of 'BDD in Action'
```

1 Adding something to the cart